

2025 RUN Spring Contest Editorial

Official Editorial

by
RUN

문제	의도한 난이도	출제자
A Game of RUN	Easy	bubbler
B Difference Maximization	Medium	ibm2006
C Lost Civilization	Medium	flappybird
D Cute Matrix	Hard	bubbler
E Minimum Spanning Arborescence	Hard	dadas08
F Red and Blue	Hard	flappybird
G Sheriruth	Hard	ibm2006
H BinaryTreefication	Challenging	mingyu331
I Testify	INSANE	ibm2006

A. Game of RUN

dynamic_programming

출제진 의도 – **Easy**

- 제출 116번, 정답 46명 (정답률 45.690%)
- 처음 푼 사람: **CgnoS**, 5분
- 출제자: bubbler

A. Game of RUN

- 서브태스크 1은 $n = 1, 2, 3$ 일 때 가능한 경우의 수를 손으로 세서 맞을 수 있습니다.
- 서브태스크 2는 3^n 가지의 가능한 돌의 배치를 모두 테스트하여 조건에 맞는 경우를 세면 맞을 수 있습니다.
- 서브태스크 3은 위의 풀이를 백트래킹으로 최적화하여 맞을 수 있습니다.

A. Game of RUN

- 마지막 칸이 빈 칸인지 백돌인지 흑돌인지, 백돌이나 흑돌이라면 현재 그룹의 왼쪽이 열려있는지 닫혀있는지로 총 5개의 상태를 정의합니다.
- 그러면 다음과 같은 식을 세울 수 있습니다.
 - E = Empty, B = Black, W = White, o = open, c = closed
- 기존 상태에 상관없이 마지막 칸을 비우면 E가 됩니다.

$$E(n) = E(n-1) + Bo(n-1) + Bc(n-1) + Wo(n-1) + Wc(n-1)$$

A. Game of RUN

- E, Bo의 끝에 흑돌을 놓으면 Bo가 됩니다.

$$Bo(n) = E(n - 1) + Bo(n - 1)$$

- Wo, Bc의 끝에 흑돌을 놓으면 Bc가 됩니다. Wc의 끝에 흑돌을 놓으면 백돌의 그룹이 조건을 위배합니다.

$$Be(n) = Wo(n - 1) + Bc(n - 1)$$

- Wo와 Wc는 대칭적으로 구할 수 있습니다.
- 최종적으로 각 DP의 n 번째 값을 구한 후 $E(n) + Bo(n) + Wo(n) \bmod 1\,000\,000\,007$ 을 출력하면 됩니다.
- Tn 이 너무 크므로 DP테이블을 전처리한 후에 각 입력에 대한 처리를 $\mathcal{O}(1)$ 에 하여야 합니다.

A. Game of RUN

- DP테이블을 매 입력마다 계산한 경우 서브태스크 4까지 통과하여 70점을 받습니다.
- 마지막 출력에서 모듈로를 빼먹었거나 올바르지 않은 모듈로 값을 사용한 경우 등, 모듈로 처리가 잘못된 경우 서브태스크 3까지 통과하여 45점을 받습니다.

B. Difference Maximization

greedy, prefix_sum

출제진 의도 – **Medium**

- 제출 110번, 정답 46명 (정답률 41.818%)
- 처음 푼 사람: **leo020630**, 10분
- 출제자: **ibm2006**

B. Difference Maximization

- Observation: 모든 0 들은 1, 혹은 M 으로 바꾸는 것이 최적입니다.
- 그렇지 않은 위치가 있었다고 하면, 해당 위치를 제외한 나머지 값들을 고정하고 나면 전체 친밀도는 해당 위치의 값에 대해 아래로 볼록한 볼록함수가 되고, 해당 볼록함수는 양 끝점에서 최댓값을 가지므로, 모순입니다.
- 0 을 제외한 수들만을 고려한 친밀도를 S 라고 하고, 0 을 제외한 수들의 합을 T , 사용된 1 의 개수를 x , M 의 개수를 Y 라고 하면, 친밀도는 $S + T(x - y) + My - x + xy(M - 1)$ 와 같이 나타낼 것입니다.
- S 는 누적합으로 계산할 수 있고, T 는 쉽게 계산 가능하므로 가능한 모든 $O(n)$ 개의 x, y 후보에 대해 답의 최댓값을 구하면됩니다.

C. Lost Civilization

constructive

출제진 의도 – **Medium**

- 제출 230번, 정답 46명 (정답률 20.870%)
- 처음 푼 사람: **속보그만보고문제풀어요**, 5분
- 출제자: flappybird

C. Lost Civilization

- 답이 있다면 항상 path graph입니다.
- N 개의 정점으로 이루어진 path graph의 각 정점의 위험도는 순서대로 $0, 1, \dots, \lfloor (N-1)/2 \rfloor, \dots, 1, 0$ 입니다.
- 각 정점을 알맞은 위치로 배정할 수 있는지 판정하면 됩니다. A_i 가 작은 것부터 끝쪽에 배정하면 됩니다.

D. Cute Matrix

ad_hoc, constructive

출제진 의도 – Hard

- 제출 192번, 정답 34명 (정답률 17.708%)
- 처음 푼 사람: **속보그만보고문제풀어요**, 17분
- 출제자: bubbler

D. Cute Matrix

- $N = 2^k$ 일 경우, 다음과 같이 구성하면 됩니다.
- $k = 0$ 이면 1×1 격자에 1을 놓으면 됩니다.
- $k \geq 1$ 이면, $N' = 2^{k-1}$ 의 답이 행렬 M 일 때 다음과 같은 답이 있습니다.

$$\begin{bmatrix} 2M & 2M - 1 \\ 2M - 1 & 2M \end{bmatrix}$$

D. Cute Matrix

- 이것이 성립하는 이유는, 모든 길이 3의 등차수열의 홀짝성은 짝짝짝, 홀홀홀, 짝홀짝, 홀짝홀 중 하나이기 때문입니다.
- 모든 홀수와 모든 짝수가 모든 행과 열에서 서로 분리되어 배치되므로 뒤쪽 2가지 경우는 불가능하고, 앞쪽 2가지 경우가 존재하려면 행렬 M 안에 등차수열이 존재하여야 하는데, 이는 M 의 귀납적 가정에 의해 역시 불가능합니다.
- 이 구성을 구현할 때에는 행렬을 재귀적으로 이어붙이는 방법도 있지만, 행 번호와 열 번호를 xor한 후 비트를 뒤집는 방법으로도 가능합니다.

D. Cute Matrix

- $N = 2^k$ 가 아니면 답은 존재하지 않습니다.
- 먼저 N 이 홀수인 경우를 살펴봅시다.
- $x = \frac{N+1}{2}$ 에 주목합니다.
- 문제의 조건에 의해, 1 과 N 이 x 를 기준으로 서로 반대쪽에 있으면 등차수열이 만들어지므로 1 과 N 은 같은 쪽에 있어야 합니다.
- 마찬가지로 2 와 $N+1, \dots, x-1$ 과 $x+1$ 의 모든 쌍은 x 를 기준으로 같은 쪽에 있어야 합니다.
- 그런데 이렇게 되면 x 의 왼쪽과 오른쪽에 오는 수의 개수가 모두 짝수여야 합니다. 그러나 귀여운 행렬을 구성하기 위해서는 왼쪽에서 2번째 칸에도 x 가 한 번은 와야 하므로 모순입니다.

D. Cute Matrix

- 이제 $N = 2k$ 일 때 k 가 3 이상의 홀수인 경우를 살펴봅시다.
- 이번에는 $x = k$ 에 주목합니다.
- x 를 왼쪽에서 두 번째 칸에 놓고 같은 논리를 전개하여 $x - 1$ 과 $x + 1, \dots, 1$ 과 $N - 1$ 을 짝지어 주면, x 의 왼쪽에 올 수 있는 유일한 수는 N 이 되고, N 과 x 를 제외한 모든 수는 그 둘의 오른쪽에 오게 됩니다.
- 이제 $x < y < N$ 인 y 값들에 주목합니다.
- x 와 N 의 홀짝성이 다르므로, y 는 항상 x 나 N 중 하나와 홀짝이 같고, 따라서 그 수와 y 의 평균 z 는 정수입니다. 또한 $x < y < N$ 이므로 $x < z < N$ 이 성립합니다.

D. Cute Matrix

- z 가 y 의 왼쪽에 있으면 길이 3의 등차수열이 만들어지므로, z 는 y 의 오른쪽에 있어야 합니다.
- 그런데 이러한 관계성은 범위 내의 모든 y 값에 대해 존재하므로, 이러한 y 값들을 정점으로 하고 y 에서 z 로 가는 간선을 포함하는 그래프는 함수형 그래프가 됩니다.
- 함수형 그래프에는 반드시 사이클이 하나 이상 존재하며, 이는 곧 “오른쪽에 있다”의 관계가 순환함을 의미하므로 모순입니다.
- $N = 2^e k, e \geq 2$ 일 경우는 $N = 2k$ 의 경우에서 모든 x 와 y 의 값에 2^{e-1} 을 곱한 것을 고려하면 같은 논리를 통해 모순을 보일 수 있습니다.

E. Minimum Spanning Arborescence

combinatorics, graph_theory

출제진 의도 – Hard

- 제출 83 번, 정답 26명 (정답률 31.325%)
- 처음 푼 사람: **속보보고문제그만풀어요**, 49분
- 출제자: **dadas08**

E. Minimum Spanning Arborescence

- Minimum Spanning Arborescence를 만드려면, 각 정점마다 자신으로 들어오는 간선들 중 가장 가중치가 작은 것을 택하면 됩니다.
- 즉, 각 정점에 대해 자신으로 들어오는 간선들 중 최소 가중치의 합을 구하면 됩니다.

E. Minimum Spanning Arborescence

- 기댓값의 선형성에 의해, 각 정점마다 독립적으로 기댓값을 구해 합하면 됩니다.
- 어떤 정점의 indegree를 k 라 하면, 자신으로 들어오는 간선들 중 최소 가중치의 기댓값은 다음과 같이 계산됩니다.
 - (모든 간선이 1 이상일 경우의 수) + (모든 간선이 2 이상일 경우의 수) $\dots / K^k$
 - $= K^k + (K - 1)^k + (K - 2)^k \dots / K^k$
 - $= 1^k + 2^k \dots + K^k / K^k$
- 위의 값을 naive하게 모든 정점마다 구하면 $O(NK \log M)$ 입니다.

E. Minimum Spanning Arborescence

- 모든 정점의 indegree의 합은 M 이므로, 서로 다른 indegree값은 최대 $O(\sqrt{M})$ 개 입니다.
- 그러므로 실제로 등장한 indegree값들에 대해서만 계산해주면 $O(N + K\sqrt{M}\log M)$ 이 됩니다.

F. Red and Blue

constructive, geometry

출제진 의도 – **Hard**

- 제출 132번, 정답 21명 (정답률 15.909%)
- 처음 푼 사람: **CgnoS**, 47분
- 출제자: flappybird

F. Red and Blue

- 나무를 자를 때는 격자를 따라 잘라야 하므로, 작업 영역의 경계를 제외한 격자를 이루는 $N - 1$ 개의 가로줄과 $M - 1$ 개의 세로줄이 있다면, 나무를 자르는 선은 전부 이 가로줄과 세로줄에 포함됩니다.
- 서로 다른 가로줄 또는 세로줄을 자르는 것을 독립이며, 어떤 줄에 포함되게 자르는 것은 다른 줄의 어떠한 요소에도 영향을 주지 않습니다.

F. Red and Blue

- 각 가로줄은 길이가 1 인 선분 M 개로 나눌 수 있고, 각 세로줄은 길이가 1 인 선분 N 개로 나눌 수 있습니다.
- 각 선분은 다음 3 가지 중 하나로 분류할 수 있습니다.
 - 두 개의 '#' 사이에 있는 선분은 절대로 잘라서는 안됩니다.
 - '#'와 '.' 사이에 있는 선분은 반드시 잘라야만 합니다.
 - 두 개의 '.' 사이에 있는 선분은 잘라도 자르지 않아도 상관 없습니다.
- 앞으로 위의 3 가지 선분을 순서대로 A, B, C 로 칭합니다.

F. Red and Blue

- 첫 번째 방법으로 잘라 만든 절단면은 언제나 두 번째 방법으로 자르는 것으로도 만들 수 있습니다.
- 첫 번째 방법으로 자르는 것이 두 번째 방법으로 같은 절단면을 만드는 것보다 힘을 덜 소모하는 경우가 있으므로 어떤 상황에서 첫 번째 방법으로 자를 수 있는지, 또 어떤 상황에서 그렇게 자르는 것이 이득인지 알아보시다.

F. Red and Blue

- 어떤 B 가 같은 줄에 있는 두 A 를 잇는 선분에 포함된다면, 다시 말해 두 A 사이에 존재한다면 그 B 를 포함해서 첫 번째 방법으로 자르는 것은 불가능합니다.
- 위 조건에 해당되지 않으며 한 번도 자르지 않은 모든 B 에 대해, 해당 B 를 포함하며 어떤 A 도 포함하지 않게 첫 번째 방법으로 최대 몇 개의 B 를 자를 수 있는지 확인합니다. 자르는 반직선이 반드시 해당 B 에서 시작할 필요는 없으므로 해당 B 에 대해 한쪽 방향만 확인하지 않도록 주의합니다.
- 그 방법으로 F 를 초과한 개수의 B 를 자를 수 있다면 첫 번째 방법으로 자르는 것이 이득이므로 첫 번째 방법으로 자르도록 합니다.

F. Red and Blue

- 첫 번째 방법으로 자르는 것이 불가능하거나 이득이 되지 않는 모든 B 를 두 번째 방법으로 자릅니다.
- 두 번째 방법으로 C 를 자를 경우 불필요한 힘이 소모되므로 잘라서는 안됩니다.
- 첫 번째와 두 번째 방법으로 자르는 모두에 대해 하나의 B 를 두 번 이상 세지 않도록 주의합니다.

F. Red and Blue

- $N - 1$ 개의 가로줄에 대해 각각 M 개의 선분을 확인하고, $M - 1$ 개의 세로줄에 대해 각각 N 개의 선분을 확인해야 합니다.
- 각 선분은 한 번 확인한 후에는 다시 확인하지 않으므로 시간복잡도는 $O(NM)$ 이 됩니다.

G. Sheriruth

ad_hoc, constructive

출제진 의도 – **Hard**

- 제출 143번, 정답 7명 (정답률 4.895%)
- 처음 푼 사람: **트랄라레로 트랄랄라이트온**, 190분
- 출제자: ibm2006

- $m \geq 2n - 3$ 이라면, 방어자를 선택하면 이길 수 있습니다. 이진수열에서 자신의 바로 뒤에 1 이 오지 않는 0 들에 대해서, 해당 0 들을 2 로 바뀌주는 전략을 생각합니다.
- 각 이진수열들은 적어도 하나의 01 을 가지고 있기 때문에, 적어도 3 개 이상의 0 은 2 로 바뀌지 않게 됩니다.
- 임의의 인덱스 i 에 대해서, 3 개의 이진수열중 적어도 하나는 i 번 원소와 $i + 1$ 번 원소가 전부 0 이기 때문에 i 번 원소가 2 로 바뀐 이진수열은 적어도 하나 이상 존재합니다.

- 이제, $m < 2n - 3$ 인 경우를 생각합시다. i 번째 비트만 1 이고, 나머지 비트는 전부 0인 수를 e_i , 라고 표기하겠습니다.
- 어떤 i, j 에 대해서, B_{e_i} 의 j 번째 원소가 0이라면 i 에서 j 로 가는 간선을 이은 그래프를 생각합시다.
- 어떤 정점 v 의 indegree가 2 이상이라면, v 를 향하는 두 정점 x, y 에 대해서 $e_x, e_y, 2^n - 1 - e_x - e_y$ 가 답이 됩니다. 이제 모든 정점의 indegree가 2 이하인 경우를 생각하겠습니다.

- 모든 정점의 indegree가 2 이하이므로, 전체 간선의 개수는 n 이하입니다.
- 만약 어떤 정점 x 의 outdegree가 0 이라면, 다른 outdegree가 1 이하인 정점 y 를 찾고 나면 $e_x, e_y, 2^n - 1 - e_x - e_y$ 가 답이 됩니다. 따라서 모든 정점의 outdegree가 정확히 1 이고, 자연스럽게 모든 정점의 indegree 또한 1 인 상황만을 고려하면 됩니다.
- 이런 경우에는 아무 정점 x 와, 해당 정점이 향하는 다른 정점 y 를 잡고 $e_x, e_y, 2^n - 1 - e_x - e_y$ 를 고려해보면, 2의 개수가 $2n - 3$ 이상이어야 함을 알 수 있습니다. 따라서 모든 케이스에 대해서 조건을 만족하지 않는 정수 세 쌍을 찾을 수 있습니다.

H. BinaryTreefication

ad_hoc, constructive

출제진 의도 – Challenging

- 제출 83번, 정답 2명 (정답률 3.614%)
- 처음 푼 사람: 韩国小南娘, 174분
- 출제자: mingyu331

H. Binary Treeification

- 모든 정점의 거리를 다르게 만들어 봅시다.
- T' 에서 모든 정점의 거리가 다르므로 조건을 만족합니다.

H. Binary Treeification

- T' 의 형태를 선형 트리로 고정하고, T 의 각 정점을 T' 위에 배치하는 형태로 해봅시다.
- Subtask 1(4점): i 번째 정점을 2^i 번 위치에 배치합니다.
- Subtask 2(8점): i 번째 정점을 $i^2 \cdot (2N + 1) + i$ 번 위치에 배치합니다.
 - i 번 정점과 j 번 정점의 거리는 $(i^2 - j^2) \cdot (2N + 1) + (i - j)$ 이고, $i - j$ 는 $-N$ 이상 N 이하이므로 주어진 거리에 대해서 해당하는 정점의 $i - j$ 와 $i^2 - j^2$ 를 얻을 수 있습니다.
이로부터 두 정점의 순서쌍을 계산할 수 있으니 모든 정점 쌍의 거리는 달라야 합니다.
- Subtask 3(11 점): $p = 101 > N$ 에 대해, i 번째 정점을 $(i^2 \bmod p) \cdot (2p + 1) + i$ 번 위치에 배치합니다.
 - 같은 방식으로 성립합니다.

H. Binary Treeification

- 모든 정점 쌍 사이의 거리를 다르게 만들려면 정점간의 최대 거리가 정점 쌍 개수인 $N(N - 1)/2$ 이상이어야 합니다.
- 고로, 트리의 지름의 길이가 $N(N - 1)/2$ 이상이어야 하고, 문제를 해결하지 못합니다.
- 즉, 원래 트리의 거리 관계를 고려하여 트리를 이진 트리로 변환하여야 합니다.
- 어떤 k 에 대해서 원래 트리의 거리를 k 배 늘린 형태로 거리가 나오도록 새 트리를 구성해 봅시다.

H. Binary Treeification

- Subtask 4(22점): 모든 부모와 자식 사이의 간선의 거리가 k 가 되도록 새 트리를 구성합니다.
- 트리의 최대 차수가 2^k 이하인 경우, 깊이 k 의 완전 이진 트리를 부모 노드에 붙여 각 리프 노드에 자식 노드들을 배정하는 방식으로 구성할 수 있습니다.
- 부모-자식 관계가 아닌 경로의 경우에는 경로 내의 최대 공통 조상 노드를 거치지 않고 이 노드에 붙인 완전 이진 트리 내부를 지나 거리가 정확히 k 배가 되지 않는 문제가 생깁니다.
- 이때 원래 거리의 k 배에서 감소하는 거리는 0 이상 $2 \log_2(\max_v(d_v))$ 이하입니다.
- $k \geq 2 \log_2(\max_v(d_v))$ 이면 k 가 최대로 감소할 수 있는 거리보다 커 거리가 겹치지 않게 구성할 수 있습니다.
- $N \leq 1000$ 일 때, $k = 20$ 에서 성립합니다.

H. BinaryTreefication

- Subtask 5(55점): 원래 거리에 비해 감소하는 거리의 길이는 짝수임을 관찰할 수 있습니다. 고로, k 가 $\log_2(\max_v(d_v))$ 이상인 홀수 정수인 경우에도 거리가 겹치지 않게 된다는것을 관찰할 수 있습니다.
- $k = 11$ 으로 두면 $N \leq 2\,000$ 에 대해 문제를 해결할 수 있습니다.

I. Testify

ad_hoc, constructive, geometry

출제진 의도 – **INSANE**

- 제출 33번, 정답 0명 (정답률 0%)
- 처음 푼 사람: **없음**, ?분
- 출제자: ibm2006

I. Testify

- 일단, 각 구역에 대해서, 해당 구역의 "높이"를 자신의 아래에 있는 직선의 개수로 정의합니다. 히카리는 각 구역의 높이를 해당 구역의 두 번째 정수로 설정할 것입니다.
- 기본적인 전략의 흐름은, 타이리츠가 특정한 높이에 있는 모든 구역을 순회하도록 하는 것입니다.

I. Testify

- Definition. i -chain이라는 것을, 높이가 i 인 영역들을 x 축방향을 기준으로 정렬한 배열이라고 하겠습니다. 어떤 i -chain의 원소 x 에 대해서, x 와 인접한 $i + 1$ -chain의 원소중 맨 오른쪽에 있는 것을 $up(x)$ 와 같이 표기하고, x 와 인접한 $i - 1$ -chain의 원소중 맨 오른쪽에 있는 것을 $down(x)$ 라고 하겠습니다.
- Observation. 어떤 i -chain의 원소 x 와 바로 뒤에 있는 원소 y 를 생각하겠습니다. $up(down(x)) = y$ 혹은 $down(up(x)) = y$ 가 성립합니다.
- 이때, x 가 up 이라는 것은 $up(down(x)) = y$ 가 성립한다는 것이고, $down$ 이라는 것은 그렇지 않은 경우라고 하겠습니다. $i + 1$ -chain의 원소 z 에 대해서, 어떤 up 인 i -chain의 원소 x 가 존재해서 $up(x) = z$ 라면 z 를 i -usable하다고 하고, $i - 1$ -chain의 원소 z 에 대해서 어떤 $down$ 인 i -chain의 원소 x 가 존재해서 $down(x) = z$ 라면 z 를 i -usable하다고 하겠습니다. 이외의 다른 모든 $i + 1, i - 1$ -chain의 원소들은 i -useless라고 하겠습니다.

I. Testify

- 이제, 히카리의 전략을 서술하겠습니다.
- i -chain의 원소들에 대해서, 다음과 같은 함수 f 를 정의합니다: i -chain의 맨 앞에 있는 원소 x 에 대해서, 해당 원소가 up 이라면 $f(x) = 1$, 아니라면 $f(x) = 0$ 으로 정의합니다. 이제, i -chain의 맨 앞에 있지 않은 원소 x 에 대해서, 해당 원소가 이전의 원소 prv 와 $up/down$ 여부가 같다면 $f(x) = f(prv)$, 아니라면 $f(x) = (f(prv) + 1) \bmod 3$ 으로 정의합니다.
- 히카리는, i -chain의 맨 앞에 있지 않은 원소 x 에 대해서 $(i \% 4) / 2, f(x), i + 1$ -usable한지의 여부, $i - 1$ -usable한지의 여부를 첫 번째 정수에 인코딩합니다.

I. Testify

- i -chain의 맨 앞쪽 원소 $f(x)$ 에 대해서는, 만약 $i = n$ 이라면 28을 첫 번째 정수로 설정합니다.
 $i = 0$ 이라면 27로 설정합니다.
- 그 외의 경우에는 $25 + f(x)$ 로 인코딩합니다.

I. Testify

- 이제, 타이리츠가 높이 i 인 구역을 전부 순회하는 방법을 서술합니다.
- 높이가 001100110011...과 같이 인코딩되었으므로, 타이리츠는 위쪽, 혹은 아래쪽으로 이동할 수 있습니다. 만약 맨 끝으로 도달한 곳이 맨 위쪽이라면, 반대로 이동해서 맨 아래쪽 구역에서 시작하도록 합니다.
- 왼쪽의 25에서 28 사이의 정수가 매겨진 구역들을 타고 올라가서 i 번째 높이까지 이동합니다.

I. Testify

- 귀납적으로 i -chain의 모든 원소를 순회하는 방법을 서술하겠습니다.
- 맨 처음에, 구역의 값은 25 혹은 26 중 하나인데, 이것으로 다음으로 이동해야 할 곳이 $i + 1$ -chain인지, 혹은 $i - 1$ -chain인지 알아낼 수 있습니다. 그 뒤 계속 그 다음 i -chain의 원소로 이동하더라도, f 값이 변화하는지를 기준으로 다음으로 이동할 방향이 위쪽인지, 혹은 아래쪽인지 관리할 수 있습니다.
- 직관적으로 보았을 때, 현재 있는 i -chain의 원소 x 에서 원하는 방향에 있는 usable한 구역은, 직전에 지나온 구역을 제외하면 유일하게 정해짐을 확인할 수 있습니다. 따라서 i -chain에서 $i + 1$ -chain, 혹은 $i - 1$ -chain으로 이동할 수 있습니다.

I. Testify

- 다음으로, $i + 1$ -chain 혹은 $i - 1$ -chain에서 i -chain으로 이동하는 스텝만이 남았습니다. 두 경우가 비슷하므로, $i + 1$ -chain의 어떤 원소 x 에서 i -chain으로 이동하는 경우만을 다루겠습니다.
- x 와 인접한 i -chain의 원소가 2개 이하라면, 걱정할 것이 없습니다.
- x 와 인접한 i -chain의 원소가 3개 이상이라면, 항상 직전에 있었던 원소는 인접한 원소들 중 뒤에서 두 번째 위치에 있었던 원소였을 것입니다. 이를 prv 라고 하고, 맨 뒤의 원소를 nxt 라고 하겠습니다. prv 이전의 모든 x 와 인접한 i -chain의 원소들은 $down$ 이었을 것이고, prv 는 up 이었을 것입니다. 따라서, prv 이전의 원소들은 $f(prv) - 1$ 을 f 의 값으로 가졌을 것이고, $f(nxt)$ 는 $f(prv)$ 보다 최대 1만큼 컸을 것입니다. 따라서, x 와 인접한 원소들중 nxt 로 특정되는 구역은 유일하게 존재합니다.

I. Testify

- 따라서, 앞서 말한 전략대로 모든 i -chain의 원소를 순회할 수 있고, 이를 통해 전체 문제를 해결할 수 있습니다.